

CC3001 Algoritmos y Estructuras de Datos

Pauta Control 1

Pregunta 1

Considere la siguiente ecuación de recurrencia:

$$T(n) = 5T(n-1) - 6T(n-2), T(0) = 0, T(1) = 1$$

- i. (1 pto.) Escriba en Java un método recursivo que calcule $T(n)$ para un n dado.

Solución:

```
int computeT(int n)
{
    if (n==0) return 0;
    if (n==1) return 1;
    return 5*computeT(n-1)-6*computeT(n-2);
}
```

- ii. (2 ptos.) Analice cuántos llamados recursivos debe realizar el método `computeT` para calcular $T(n)$. Para esto, defina la ecuación de recurrencia correspondiente al método y luego resuélvala. Es suficiente con encontrar el orden del número de llamados recursivos.

Solución:

Ecuación de recurrencia:

$$F(n) = F(n-1) + F(n-2), F(0) = 1, F(1) = 1$$

Esta ecuación es igual a la ecuación de Fibonacci pero con condiciones iniciales distintas. Su complejidad es igual, $O(\phi^n)$ (se obtiene a través del método del polinomio característico).

- iii. (2 ptos.) Resuelva la ecuación de recurrencia de $T(n)$ en forma exacta.

Solución:

Usando polinomio característico:

$$\lambda^n = 5\lambda^{n-1} - 6\lambda^{n-2} \Rightarrow \lambda^2 - 5\lambda + 6 = 0$$

$$\lambda_1 = 3, \lambda_2 = 2 \Rightarrow T(n) = A3^n + B2^n$$

Por condiciones iniciales:

$$A = 1, B = -1 \Rightarrow T(n) = 3^n - 2^n$$

- iv. (1 pto.) Utilice la solución encontrada en la parte (iii) y escriba un método iterativo que calcule $T(n)$. ¿Qué complejidad tiene este algoritmo?

Solución:

```
int computeT(int n)
{
    return potencia(3,n)-potencia(2,n);
}
```

Calcular potencia toma tiempo $O(\log n)$, por que el método toma tiempo $O(\log n)$.

Forma alternativa de resolverlo: utilizar programación dinámica (como en Fibonacci). En ese caso el algoritmo toma tiempo $O(n)$.

Pregunta 2

Diseñe un algoritmo y escriba el método

```
boolean correcto(char[] x)
```

que retorna *true* si en el arreglo x viene correctamente escrita una lista de números enteros con el siguiente formato: $(n_1, n_2, n_3, \dots, n_k)$. La cantidad de números en la lista es variable. Son válidas las listas “()” (lista vacía) y “(n_1)” (lista con un sólo número). Un número entero se define como una secuencia de dígitos. No deben venir espacios entre medio.

Resuelva este problema en los dos pasos siguientes:

- i. Dibuje el diagrama de estados correspondiente a su algoritmo.
- ii. Escriba el código de su programa.

Solución

```
public class ParseNums{

    public static boolean parse(char[] s) {
        if (s.length < 2 || s[0] != '(') return false;
        int i = 1, n = s.length;
        //invariante: el texto hasta la posicion i es correcto
        while (i < n - 1) {
            int desde = i;
            while (i < n - 1 && esNumero(s[i])) i++;
            //no avanza nada, es incorrecto porque debe haber al menos un numero
            if (i == desde)
                return false;
            //luego del numero si i < n - 2 debe venir una , (si no debe salir)
            if (i < n - 2 && s[i] == ',') i++;
        }
        //el ultimo char debe ser un )
        return i == n - 1 && s[i] == ')';
    }

    public static boolean esNumero(char c) {
        return c >= '0' && c <= '9';
    }

    public static void main(String[] args) {
        comprobar(true, "()", "(1)", "(123)", "(1,234)", "(1,211,99,0)");
        comprobar(false, "(", "(", "(", "(", "(1", "(1,", "(000,",
            "(000,)", "(000,,)", "(000,,", "(000))");
    }

    public static void comprobar(boolean esperado, String... strings) {
        for (String s : strings) {
            if (parse(s.toCharArray()) != esperado)
                throw new RuntimeException("incorrecto para " + s);
        }
    }
}
```

Pregunta 3

Suponga que se dispone de la siguiente implementación de `Nodo` de una lista enlazada:

```
class Nodo{
    int valor;
    Nodo sgte;
}
```

Diseñe un algoritmo iterativo y escriba el método

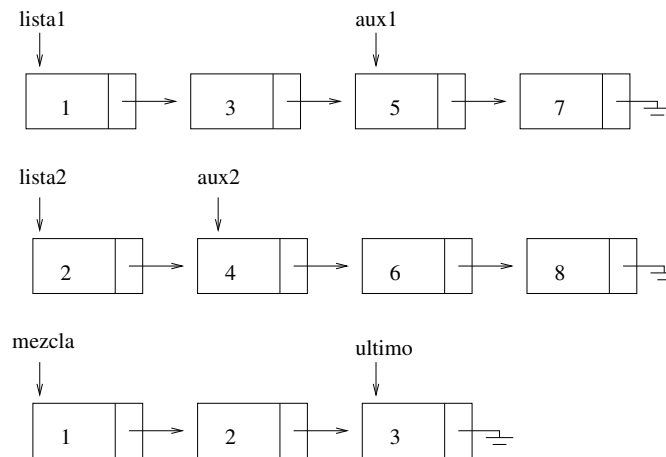
```
Nodo mezclarListasOrdenadas(Nodo Lista1, Nodo Lista2)
```

que recibe como parámetros dos listas *ordenadas* (ascendentemente por *valor*), y retorna una lista correspondiente a la mezcla ordenada de ambas listas. Explique formalmente cuál es el invariante del algoritmo (puede ayudarse con una figura), la inicialización, la condición de término, en qué parte del código se rompe el invariante, dónde se recupera, etc.

Solución:

Invariante: **res** contiene el resultado de mezclar las listas hasta antes de **aux1** y **aux2**.

Figura:



Seudocódigo para listas sin nodo cabecera:

```
Nodo mezclarListasOrdenadas(Nodo lista1, Nodo lista2)
{
```

```

// inicializacion
Nodo aux1=lista1, aux2=lista2, mezcla, ultimo;
if (aux1.valor<aux2.valor)
{
    mezcla=new Nodo(aux1.valor,null);
    aux1=aux1.sgte;
}
else
{
    mezcla=new Nodo(aux2.valor,null);
    aux2=aux2.sgte;
}
ultimo=mezcla;
// iterar hasta que alguna de las listas se acabe
while (aux1!=null && aux2!=null)
{
    if (aux1.valor<aux2.valor)
    {
        Nodo n=new Nodo(aux1.valor, null);
        aux1=aux1.sgte; // rompe invariante
    }
    else
    {
        Nodo n=new Nodo(aux2.valor, null);
        aux2=aux2.sgte; // rompe invariante
    }
    ultimo.sgte=n; // recupera invariante
    ultimo=ultimo.sgte;
}
// agregar el resto de la lista que aun tiene nodos
Nodo aux=aux1==null?aux2:aux1;
while (aux!=null)
{
    Nodo n=new Nodo(aux.valor, null);
    ultimo.sgte=n;
    aux=aux.sgte;
    ultimo=ultimo.sgte
}
return mezcla;
}

```

Seudocódigo para listas con nodo cabecera:

```
Nodo mezclarListasOrdenadas(Nodo lista1, Nodo lista2)
{
    Nodo aux1=lista1.sgte, aux2=lista2.sgte, mezcla=new Nodo(0, null), ultimo=mezcla;
    while (aux1!=null && aux2!=null)
    {
        if (aux1.valor<aux2.valor)
        {
            Nodo n=new Nodo(aux1.valor, null);
            aux1=aux1.sgte;
        }
        else
        {
            Nodo n=new Nodo(aux2.valor, null);
            aux2=aux2.sgte;
        }
        ultimo.sgte=n;
        ultimo=ultimo.sgte;
    }
    Nodo aux=aux1==null?aux2:aux1;
    while (aux!=null)
    {
        Nodo n=new Nodo(aux.valor, null);
        ultimo.sgte=n;
        aux=aux.sgte;
        ultimo=ultimo.sgte;
    }
    return mezcla;
}
```

También lo pueden hacer sin crear nodos nuevos.